FlashAttention & PyTorch SDPA

Tiling, online softmax, memory-efficient attention and GPU memory traffic

Detailed study notes (beginner level) — Module 3, KV cache series

Class date	17 October 2026 (recording started 14:32:30 GMT)
Tutor	Paul
Format	Online class, about 3 hours (transcript timestamps 00:05:46 – 03:07:35)
Level / style	Beginner / detailed

Materials covered

File	What it is
GMT20261017-143230_Recording_transcript.vtt	Zoom transcript of the class (main source)
FLASH___SDPA.pdf	Tutor's handwritten iPad notes, 13 pages (referred to as PDF p1–p13)
08_flashattention.md	Course notebook: FlashAttention concept via SDPA backends (NB 8)
09_pytorch_sdpa.md	Course notebook: migrating to F.scaled_dot_product_attention (NB 9)
10_pagedattention_vllm.md	Course notebook: PagedAttention / vLLM (NB 10), not taught in this class

Timestamps in [brackets] refer to the transcript. [unclear: ___] marks words the speech-to-text made unreliable. Colour boxes: Key Concept, Example, Tip, Warning, Q&A.

Table of contents

0. Class structure and topic map 4

0.1 What happened, in the order it happened 4

0.2 Topic map 4

1. Where FlashAttention fits: recap of KV cache, MQA/GQA, MLA 5

2. Attention and the N² memory problem 7

2.1 The formula (simple, then precise) 7

2.2 Counting the scores 7

2.3 Why this is a problem 7

3. GPU memory and hardware basics 9

3.1 Parts of a GPU (as explained) 9

3.2 H100 vs L4 (table the tutor generated live) 9

3.3 Where the traffic comes from 9

4. FlashAttention: tiling 11

4.1 The 8-token worked example (tutor's) 11

4.2 The algorithm in words 11

4.3 Versions, hardware and the people behind it 12

5. Online softmax ("wait and update") 13

5.1 How it works step by step (tutor's description) 13

5.2 The running state and its terms 13

6. PyTorch SDPA internals: memory-efficient attention (chunking) 15

6.1 Step A: break only the queries 15

6.2 Step B: shapes (d = 4) 15

6.3 Step C: break Q, K and V 15

7. The PyTorch SDPA API: backends, FA vs SDPA, causal flag 17

7.1 Three backends 17

7.2 FlashAttention vs PyTorch SDPA (MEA) 17

7.3 The Hugging Face switch 17

7.4 Causal (decoder) attention 18

8. Demos and benchmarks (Colab) and installing FlashAttention 19

8.1 FlashAttention demo notebook 19

8.2 SDPA notebook 19

8.3 Installing the flash-attn package (what went wrong) 20

9. Practical guidance and Q&A (live questions) 21

9.1 Nilesh: how deep to go on the maths? [02:58:14–03:01:59] 21

9.2 sridhar k: production inference and training [03:02:01–03:07:21] 21

9.3 Kernels and the wider ecosystem (tangents) 21

10. Key takeaways 22

11. Glossary 23

12. Action items, logistics, resources 24

12.1 Homework and next steps 24

12.2 Schedule and logistics (as stated) 24

12.3 Resources, links and tools mentioned 24

13. Unresolved or unclear points 25

Appendix A. PagedAttention / vLLM (from NB 10 only; not taught in class) 26

Quick revision cheat sheet 27

0. Class structure and topic map

The tutor jumped between topics, so these notes are organised by **topic in logical order** (prerequisites first), not by transcript order. This section shows how the class actually ran and where each topic appears.

0.1 What happened, in the order it happened

Time	Type	Content
00:05:46–00:12:42	Admin	Audio check, LangChain/poll chat, vLLM question, repo push (git pull), diagram-tool suggestions
00:12:42–00:19:52	Recap	Goal of today; KV cache/PA, MQA/GQA, MLA recapped as "what each tried to do"
00:19:52–00:24:31	Explanation	Core idea: break Q, K, V into blocks (tiling in FA, chunking in SDPA); not about accuracy
00:24:39–00:37:27	Explanation + questions	Attention cost N^2, memory numbers, doubling rule, 32K/1M, OOM
00:37:21–00:50:43	Explanation	FlashAttention tiling worked example (S11, S12, S21, S22)
00:50:52–01:03:27	Explanation + questions	Online softmax; FA versions/hardware; tiles discarded; backprop; HF attention setting
01:03:27–01:19:11	Explanation	HBM traffic in standard vs flash attention; GPU architecture; H100 vs L4 table
01:19:11–01:34:35	Demo	Colab: manual vs flash attention (T4 fails, L4 works), speed, memory, sequence sweep, OOM
01:34:36–02:01:44	Break, demo, tangent	Parallel tiles; PyPI package; HF code; flash-attn install attempts (failed in Colab); admin chat
02:01:57–02:37:35	Explanation	Memory-efficient attention (chunking) with 8×4 shapes; running softmax terms
02:37:39–02:48:54	Explanation + demo	FA vs SDPA differences; three backends; SDPA Colab benchmark; causal flag
02:48:54–02:57:58	Wrap-up + tangents	Notebook links, Notion, softmax question, logistics, PyTorch conference chatter
02:58:07–03:07:35	Q&A	Nilesh (how deep to go), sridhar k (production, kernels, newer attention variants); close

0.2 Topic map

"Study order" is the order recommended for a learner; "Taught at" lists every place the tutor touched the topic.

Study	Topic	Taught at (transcript)	Materials
1	Where FlashAttention fits (recap of KV cache, MQA/GQA, MLA)	00:08:17–00:24:31	PDF p1; NB 8/9/10 recaps
2	Attention and the N^2 memory problem	00:24:39–00:37:27	PDF p2–3; NB 8, NB 9 code
3	GPU memory and hardware basics	00:54:36, 01:03:27–01:19:11	NB 8 (HBM vs SRAM)
4	FlashAttention: tiling	00:19:52–00:50:43, 01:06:49–01:09:50, 01:35:46	PDF p3–5, p12; NB 8 diagram
5	Online softmax	00:50:52–00:55:40, 00:58:37–01:00:30, 02:24:52–02:37:35, 02:52:36–02:55:09	PDF p6, p12; NB 8
6	PyTorch SDPA internals: memory-efficient attention (chunking)	00:58:05, 02:01:57–02:37:35	PDF p6–p12; NB 9
7	The SDPA API: backends, FA vs SDPA, causal flag	01:31:20, 02:37:39–02:44:15	PDF p13; NB 8 diagram, NB 9

Study	Topic	Taught at (transcript)	Materials
8	Demos and benchmarks (Colab), installing FlashAttention	01:19:11–01:34:35, 01:36:12–02:01:44, 02:44:27–02:48:54	NB 8 benchmark
9	Practical guidance and Q&A (depth, hardware, kernels, newer variants)	00:56:17, 01:42:18, 02:58:14–03:07:21	—
—	Logistics and tangents	00:06:11–00:13:34, 01:44:14–01:48:09, 02:50:10–02:57:45	—
—	PagedAttention / vLLM	only 00:07:00 (deferred to deployment module)	NB 10 only

1. Where FlashAttention fits: recap of KV cache, MQA/GQA, MLA

Summary. Every earlier technique changed *what* gets stored or reused. Today's two techniques (FlashAttention and PyTorch SDPA) change *how attention is executed on the GPU*. They do not improve accuracy; they manage GPU memory traffic. [00:14:38–00:15:21]

Technique	Main goal, in the tutor's words	Timestamp
KV cache / PagedAttention ("PA")	Reuse previously calculated K and V instead of recalculating. (Cost, memory and speed benefits follow, but "avoid recalculation" is the main idea.)	[00:15:31–00:16:35]
MQA / GQA	Store fewer key and value heads ("reduce KV heads, that is the entire game"). KV-cache reduction follows automatically.	[00:16:57–00:18:09]
MLA	Represent the cache more compactly (compression).	[00:18:14–00:18:58]
FlashAttention / SDPA	Can we execute attention with less memory? This is GPU traffic management.	[00:19:04–00:20:11]

KEY CONCEPT — Not about accuracy

Both algorithms of today "do not help you in increasing the accuracy of your model". The formula is unchanged; the work is only **broken into smaller pieces** so the GPU moves less data. The tutor stressed this several times [00:14:44–00:15:21, 00:24:06–00:24:31].

Unlike MQA/GQA/MLA, this is not about building a smaller model. It is about **executing** the model: when training, how to get faster speeds by optimising GPU data movement. It also helps during inference [00:24:17, 01:31:53].

TIP — Prerequisite and mindset

You need to know scaled dot-product attention well; it is "the core fundamental, even for today's class" [00:08:23].

Practically it is "just one line of code" but "behind that one line, really a lot of things go ahead" [00:08:54–00:08:59].

Of the two algorithms, FlashAttention is the core idea. SDPA uses a variant called memory-efficient attention [00:08:38].

Q&A

Q: Krishna (first class): is there anything I need to know before today's session?

A: A little: at least scaled dot-product attention, which is the fundamental even for today. [00:08:17]

Q: Participant (name garbled, "Unashi"; question inferred from the answer): how is this related to the MQA/GQA/MLA reductions?

A: It is a totally different context. Those reduce KV heads or cache size; this is about the execution of the model and how data moves on the GPU. [00:24:00–00:24:31]

Q: Kumara Dev (question partly inaudible): about the vLLM / PagedAttention part.

A: PagedAttention was the main core part; the vLLM practical will come in the deployment module because it needs a GPU and a setup. [00:06:51–00:07:13]

Material reference: *PDF p1 ("Today's Agenda": Flash-Attention and PyTorch SDPA, "Accuracy ✗ / GPU Traffic"; PA/KV cache, MQA/GQA, MLA, FA one-liners). Notebook recaps: NB 8 ("every notebook so far changed what gets cached"), NB 9, NB 10.*

Additional info from materials (not said in class)

- NB 8: FlashAttention "speeds up the attention computation itself, without changing the math at all: same output, less memory traffic".
- NB 10: PagedAttention (Kwon et al., 2023, the vLLM paper) changes the **allocation strategy** of the cache buffer, not the attention math. Nuance: in class the tutor grouped "PA / KV cache" under "reuse"; the notebook describes PA as an allocation scheme (see Appendix A).
- NB 10 end-of-course recap: NB 0–1 why caching K/V matters; NB 2 cache size; NB 3–6 MHA/MQA/GQA/MLA; NB 7 RoPE and context extension; NB 8–9 fast attention computation; NB 10 cache allocation for many concurrent requests.

Said in class, not in materials

- The one-line goals above, and the statement that FA/SDPA are about hardware (HBM, SMs, L1/L2 cache) rather than model quality.

2. Attention and the N² memory problem

Summary. Attention compares every token with every other token and stores one score per pair. That table grows with the *square* of the sequence length, so long sequences exhaust GPU memory.

2.1 The formula (simple, then precise)

Simple: each token (query) looks at all tokens (keys), turns the match scores into weights, and mixes the values with those weights.

Precise: SDPA (scaled dot-product attention) = softmax($Q \cdot K^T / \sqrt{d_k}$) · V [PDF p2; 00:23:21–00:23:44].

Step	Symbol	Operation	Source
1	S	$S = Q \cdot K^T$ (scores for every query/key pair)	PDF p3; [00:38:02]
2	P	$P = \text{softmax}(S / \sqrt{d_k})$: scaling then softmax	PDF p3; [00:38:14–00:38:30]
3	O	$O = P \cdot V$	PDF p3; [00:38:30]

Here "SDPA" is the first step of the transformer block, and the tutor wants the formula "by heart" [00:22:45–00:23:50]. The name of the PyTorch API comes straight from the paper's name for this operation [00:22:24–00:22:43].

2.2 Counting the scores

EXAMPLE — Tutor's own numbers

- **N tokens → N × N scores = N² = O(N²).** With 8 tokens: 8 × 8 = **64** scores [00:25:01–00:26:06; PDF p2].
- Add batch and heads: **batch = 2, heads = 8, sequence length = 1024** → 2 × 8 × 1024 × 1024 ≈ **16 million** scores [00:26:37–00:27:35; PDF p2].
- Bytes per number: **FP16 = 2**, **FP32 = 4**. So ≈ 16 M × 2 ≈ **32 MB** (FP16) and ≈ 16 M × 4 ≈ **64 MB** (FP32). The tutor called these "rough numbers" [00:28:29–00:29:39; PDF p2].
- Sequence length 2048 (FP32) ≈ **256 MB**; 4096 ≈ **1 GB** [PDF p3; 00:30:41–00:30:48].

Sequence length (batch 2, 8 heads)	Scores in N×N matrix	FP32 memory (4 bytes)	Source
1024	≈ 16 million	≈ 64 MB (FP16: ≈ 32 MB)	PDF p2
2048	4× the above	≈ 256 MB	PDF p3
4096	4× again	≈ 1 GB	PDF p3; [00:30:35]
~8000 (tutor's extension)	4× again	≈ 4 GB (by the doubling rule)	[00:34:01]
32K	32K × 32K	too big for the GPU in one shot (no figure given)	[00:35:41–00:36:11]
1M ("recently we have reached")	—	no figure given	PDF p3; [00:30:19]

KEY CONCEPT — The doubling rule

Double the sequence length → 4× the memory for the attention scores (PDF p3: "(2T) = 4T memory"; "Double the seq. length = 4 times {attention scores}") [00:31:12–00:33:45]. T stands for sequence length; the square comes from the $Q \cdot K^T$ operation.

2.3 Why this is a problem

- Standard self-attention keeps the whole N × N matrix sitting in GPU memory at once. With many batches and heads it becomes huge, the GPU is "allocated" and cannot do other operations (softmax, multiplying with V, gradients later) [00:39:08–00:39:26, 00:54:47–00:55:20].
- For 32K × 32K scores you "cannot fit that"; the result is an out-of-memory error (**OOM**) [00:35:41–00:36:29; PDF p3].
- Memory is the issue even for training, where gradients must also be saved [01:04:37].

- Today's models use sequence lengths far beyond the original paper (the tutor: "more than 1000×" larger), so this matters much more now [00:17:39–00:17:56].

WARNING — Do not think the causal/decoder case avoids this

A participant asked whether the number would be smaller because tokens do not see all other tokens. The tutor said no: for this calculation, think of the full matrix, in both encoder and decoder, because in the first step the idea is to capture context across all tokens. [00:27:46–00:28:22]

Q&A

Q: Darshan (wording unclear): will the count actually be lower because we do not look at the other tokens?

A: No. In encoder and decoder alike, attention looks across the tokens; think of the full matrix. [00:27:46–00:28:22]

Q: Participant: can we just store the values in a smaller type, e.g. FP32 → FP16?

A: Yes, that gives about a 50% reduction, but for long sequences the numbers are still too large. [00:36:41–00:37:03]

Q: Sanjiv: can linear attention solve this?

A: It helps, but not fully, because this discussion is about the memory of the scores. [00:37:07–00:37:21]

Q: Participant: isn't it about not making the GPU wait for memory?

A: Yes, that is one part; it comes up when HBM is discussed (Topic 3). [00:36:31–00:36:41]

CONFLICT FLAGGED — HBM figure said later in class vs. handwritten notes

The PDF gives ≈ 32–64 MB for batch 2, 8 heads, sequence 1024. At [00:15:45] the tutor says that same case "was coming close to 1 GB". The notes (and the 4096 row) suggest 1 GB belongs to 4096, not 1024. Both versions are shown; the PDF figures are used above.

EXTRA CONTEXT (NOT FROM CLASS) — Arithmetic check

2 × 8 × 1024 × 1024 = 16,777,216 scores. × 4 bytes = 67,108,864 bytes ≈ 64 MiB; × 2 bytes ≈ 32 MiB. This matches the PDF.

Material reference: *PDF p2–3. NB 9 contains the same math as a function (below). NB 8 uses the same default shape (2, 8, 1024, 64).*

Additional info from materials (not said in class)

- NB 8: naive attention "materializes the full seq_len × seq_len score matrix in slow HBM". The FLOP count is roughly unchanged, while memory traffic drops dramatically, "which is what actually limits speed on modern GPUs".
- NB 9 hand-written reference attention (identical to what SDPA fuses):

```
def manual_attention(q, k, v, is_causal=True):
    d_k = q.shape[-1]
    scores = q @ k.transpose(-2, -1) / math.sqrt(d_k)
    if is_causal:                    # mask future tokens
        mask = torch.triu(torch.ones(seq, seq, dtype=torch.bool), diagonal=1)
        scores = scores.masked_fill(mask, float("-inf"))
    weights = torch.softmax(scores, dim=-1)
    return weights @ v
```

Said in class, not in materials

- The 16M-score example with byte sizes, the doubling rule (4×), the 4096 → 8000 extension, 32K and 1M discussion, FP16 vs FP32 and linear-attention answers.

3. GPU memory and hardware basics

Summary. A GPU has a large main memory (HBM) and very fast but tiny on-chip memory (SRAM, registers, caches). Standard attention keeps writing and reading the big N × N matrix to and from HBM, which is slow and wasteful. FlashAttention keeps small tiles in the fast memory instead. The tutor said this part is "more from the hardware perspective" [00:14:56].

3.1 Parts of a GPU (as explained)

Component	What the tutor said	Time
HBM (high bandwidth memory)	The main GPU memory ("VRAM" at high level). Model weights (e.g. model.safetensors) and gradients during backprop sit here. Enterprise GPUs such as H100 use HBM; consumer GPUs use GDDR (like DDR4/DDR5 in ordinary RAM); the L4 has GDDR6 technology.	[01:11:08–01:11:52, 01:18:12–01:19:03]
SM (streaming multiprocessor)	The main processing unit of the GPU; it connects the CUDA cores, registers and shared memory. H100 has about 132 SMs; L4 has fewer (the tutor was unsure: "close to 20 to 30 … I don't remember the exact numbers"). H100 comes as PCIe and SXM variants; SXM has more.	[01:13:01–01:13:51]
CUDA cores	Where the actual compute (all matmul-style operations) happens; related to SIMD (single instruction, multiple data).	[01:12:15–01:12:50]
Registers	Store values used by each individual GPU thread.	[01:11:59–01:12:15]
Shared memory / SRAM	SRAM = static RAM; fast on-chip memory, about 200 KB per SM ("close to like 200 KB"). Tiles are loaded here.	[01:07:22, 01:14:04]
L1 / L2 cache	Very fast though small; help access memory between the SMs. L2 does much of the processing-side speed-up.	[01:16:18–01:16:41]

3.2 H100 vs L4 (table the tutor generated live)

The tutor asked an AI assistant for a comparison table and showed a screenshot (the table itself is not in the supplied files). Only these values were read out [01:15:09–01:17:07]:

Property	NVIDIA H100 (SXM)	NVIDIA L4
GPU memory	80 GB (HBM; no GDDR)	24 GB (GDDR6, "DDR6 technology")
Peak memory bandwidth	≈ 3.35 TB/s ("3.35 KB per second" in the transcript: [unclear], TB/s intended since the tutor says "TBs per second")	≈ 300 GB/s
SMs	≈ 132	roughly 20–50 [unclear: tutor unsure]

KEY CONCEPT — Peak memory bandwidth is the number to look at

The tutor: "the most important part … is the peak memory bandwidth" and how much HBM is available, because everything (model, gradients) sits in HBM [01:16:47–01:18:12]. If the N × N matrix is far bigger than what the memory and bandwidth can handle, it becomes a bottleneck. With only 24 GB on an L4, long-sequence attention simply cannot execute [01:15:45–01:16:13]. Higher bandwidth helps, but the movement between SRAM and HBM is the thing to reduce [01:17:22–01:17:39].

3.3 Where the traffic comes from

Using his Excalidraw diagram (not supplied), the tutor walked through the operations [01:03:54–01:09:50]:

	Standard attention	FlashAttention
1	Input Q, K, V (in HBM); calculate QK^T	Q, K, V stay in HBM; load only **tiles** into fast on-chip memory
2	Result is the big N × N matrix, **written to HBM** (first big HBM use)	Compute a block of scores in on-chip memory
3	Read it back; scale by $1/\sqrt{d_k}$, apply softmax; **written to HBM again** (second big HBM use)	Scale and apply **online (partial) softmax** on the block

	Standard attention	FlashAttention
4	Read it back again to multiply with V	Accumulate the weighted V results
5	During backprop, gradient accumulation also needs memory	If more K/V tiles remain, go back, load the next tile, loop; write only the final output

The two orange blocks in the tutor's diagram are the two places where the big matrix hits HBM (after QK^T and after softmax). Because "write takes a lot of time, then the reading also takes a lot of time", this read/write traffic is the bottleneck [01:05:36–01:06:43, 01:09:20–01:09:48, 01:20:29].

TIP — HBM must stay free

The model weights, activations, softmax and later matmuls and gradients all need HBM, so "always free it up" rather than letting the N × N matrix sit there [00:54:44–00:55:20, 01:04:27–01:04:57].

Text diagram (recreated from NB 8; the tutor's version had the same flow):

```
NAIVE                                  FLASHATTENTION
load Q,K,V from HBM                    load Q,K,V tile into SRAM
  v                                      v
full seq x seq scores in HBM           partial scores in SRAM
  v                                      v
softmax in HBM                         accumulate softmax in SRAM
  v                                      v
write output to HBM                    write only the final output tile to HBM
```

Q&A

Q: Participant (comment inferred from the answer): is this like "unified memory"?

A: The tutor agreed with "unified memory" and said PyTorch SDPA refers to it that way too [01:09:50–01:10:06]. [unclear: the claim is not supported by the materials; see Unresolved points.]

Material reference: *Only NB 8 (HBM vs SRAM, flow diagram). The GPU component list, H100/L4 values and SM details were not in any supplied file.*

Additional info from materials (not said in class)

- NB 8: "The real bottleneck is memory bandwidth, not FLOPs"; HBM is slow global memory and SRAM is fast on-chip memory (called "IO-awareness"). FlashAttention tiles the computation into blocks that fit in SRAM and never writes the full score matrix to HBM.

Said in class, not in materials

- GDDR vs HBM, SM/CUDA core/register/shared-memory descriptions, H100 vs L4 numbers, and where model weights and gradients live.

4. FlashAttention: tiling

Summary. FlashAttention keeps the attention formula exactly as it is but cuts Q, K and V (and therefore the N × N score matrix) into small blocks called **tiles**, processing one at a time in fast memory. The tutor: "tiling is the main process through which we try to save the memory" [00:38:56].

KEY CONCEPT — Tiling

Simple: do not build one giant table of scores; build and use small pieces of it, one after another, then throw each piece away.

Precise: split Q, K, V into blocks Q_i, K_j, V_j; compute score blocks $S_{ij} = Q_i K_j^T / \sqrt{d_k}$ in fast memory; fold them into the output with online softmax (Topic 5). Same math, same final result. In SDPA the same idea is called **chunking**.

4.1 The 8-token worked example (tutor's)

Start: 8 tokens → an 8 × 8 matrix = **64** attention scores. Split into 2 × 2 blocks → **4 tiles**, each 4 × 4 = **16** scores; 4 × 16 = 64 [00:39:43–00:42:15; PDF p4].

Tile 1 $Q_1 K_1^T$ 16 scores	**Tile 2** $Q_1 K_2^T$ 16 scores
Tile 3 $Q_2 K_1^T$ 16 scores	**Tile 4** $Q_2 K_2^T$ 16 scores

Rows = query blocks (Q_1: tokens 1–4, Q_2: tokens 5–8).
Columns = key blocks (K_1: tokens 1–4, K_2: tokens 5–8).
V is split the same way: V_1, V_2.

Check: 4 tiles × 16 scores = 64 = the full 8 × 8.
Tutor: in real life the number of tiles is "much higher".

Scores for the two query blocks (scaling by $\sqrt{d_k}$ is unchanged by the split) [00:46:22–00:49:14; PDF p5]:

Block	Formula	Meaning
S_{11}	$Q_1 K_1^T / \sqrt{d_k}$	query block 1 vs key block 1
S_{12}	$Q_1 K_2^T / \sqrt{d_k}$	the same queries must also look at key block 2
S_{21}	$Q_2 K_1^T / \sqrt{d_k}$	query block 2 vs key block 1
S_{22}	$Q_2 K_2^T / \sqrt{d_k}$	query block 2 vs key block 2

Because "one query needs to look into every key", each query block meets every key block [00:47:33–00:47:48]. The V blocks are brought in afterwards [00:49:14–00:49:28].

4.2 The algorithm in words

- For each query block: process the first key block, run the softmax step, then move to the next key block and run it again, and so on for all key blocks [00:49:54–00:50:22].
- The point is "I don't want that full N × N matrix sitting on top of the GPU" [00:49:34–00:49:49].
- The tutor likened it to **divide and conquer** [00:50:22].
- Tiles are computed **separately for each input in the batch** [00:55:27].
- Tiles can be processed **in parallel**; multiple SMs / compute units take them [01:35:46–01:36:12].
- It works in **training and inference** [00:48:57, 01:31:53].
- Backpropagation is unaffected; gradients flow through Q_1, Q_2... as before [01:02:03–01:02:23].
- It does not lose the attention context: only the arithmetic is done in pieces; the context is kept in the final softmax [01:01:02–01:01:49].

TIP — Analogy the tutor used

Depthwise-separable convolution in CNNs: rather than one big operation, break it into steps so it can run on very small hardware (even a Raspberry Pi). FlashAttention does the same kind of hardware-level optimisation [01:30:38–01:31:20].

4.3 Versions, hardware and the people behind it

Item	What was said	Time
Versions	Flash Attention 1, 2, 3 (and 4 exists). **FA2 is the widely accepted, standard one.**	[00:57:00, 01:36:26]
FA3	Specialised for few GPUs: H100 / Hopper.	[00:57:07, 01:36:40]
FA4	Written for H100 and B200 (Hopper and Blackwell series); A100 is not in that series.	[01:58:08]
T4	Not supported (2018-era GPU, hardware incompatibility). Works on L4, A100, G4 ("RTX 6000") and anything newer; Hopper/Blackwell best.	[01:24:34–01:26:55]
GPU families	Ampere (e.g. A10), Hopper, Ada, Blackwell; Rubin is upcoming.	[01:25:20–01:26:55]
Authors/papers	Tri Dao ("Dao AI Lab"); every FlashAttention paper starts with standard attention, complains about it, then breaks it down (S_{11}, S_{12}...). The tutor's slides were taken from the paper and "decoded with AI".	[01:36:55–01:37:44]

WARNING — Hardware decides the algorithm

If the hardware is not there, "the selection of the algorithm really doesn't matter" [03:03:10]. Check GPU compatibility before choosing a FlashAttention version.

Q&A

Q: Nitin: is this divide and conquer?

A: Pretty much, yes. (He pointed to a reference whose name is garbled in the transcript, "station matrix" [unclear].) [00:50:22–00:50:37]

Q: Participant: are tiles calculated separately for each input in a batch?

A: Yes, for every item in the batch the tiling takes place. [00:55:27]

Q: Sush: if we have more memory, do we not need tiles?

A: No. Even with a lot of memory, longer sequence lengths are a problem; recent techniques use specialised kernels (Liger-, CUDA- or Triton-based). [00:56:06–00:56:44]

Q: Gaurav: does breaking it lose the attention mechanism?

A: No. Only the mathematical operations are done in pieces; the main context stays in the final softmax. [01:01:02–01:01:31]

Q: Abhishek: what about backpropagation?

A: No impact; it is normal and gradients flow the same way. [01:02:03–01:02:25]

Q: Murtha: do tiles run sequentially or in parallel?

A: Parallel, always; multiple SMs/compute units are used. [01:35:46–01:36:12]

Q: Participant: is SDPA the implementation of FlashAttention in PyTorch?

A: Cannot say that directly: PyTorch calls it memory-efficient attention; FlashAttention is available through its own package (and as an SDPA backend; see Topic 7). [01:00:37–01:00:53]

Material reference: *PDF p3 (S, P, O), p4 (8×8 → four 4×4 tiles), p5 (Q_1, Q_2, K, V split and S_{11}...S_{22}), p12 (FlashAttention → online softmax). NB 8 mermaid "Naive vs FlashAttention".*

Additional info from materials (not said in class)

- NB 8: the real fused CUDA FlashAttention-2 kernel benchmark lives in a separate Colab notebook (GPU required). On CPU/MPS the SDPA backends may converge or fall back to math.
- NB 8: FlashAttention is "a systems optimization, not a math change"; output is numerically identical to naive attention.

Said in class, not in materials

- The S_{11}...S_{22} walkthrough, version/hardware details (FA1–4, T4), parallel tiles, backprop and batch answers, the depthwise-convolution analogy.

5. Online softmax ("wait and update")

Summary. After $Q \cdot K^T$ comes softmax. Normal softmax needs the whole row of scores before it can normalise. The **online softmax algorithm** instead keeps running totals and updates the result as each block arrives, so the full N × N matrix never has to exist. The tutor called it "the most important part" [00:51:15].

KEY CONCEPT — Online softmax

Simple: do not wait for all scores. Take one block, compute a *partial* softmax, keep a small running state, then take the next block and update the state. The tutor's phrase: **"wait and update"** [00:52:12–00:52:23].

Precise (as taught): the result is the equivalent of the *global* softmax, computed incrementally, tile by tile. After a tile's contribution has been folded in, that tile's information can be removed from the system; only the later V blocks are still needed [00:52:36–00:53:12, 01:00:03–01:00:37].

5.1 How it works step by step (tutor's description)

- Standard softmax works on the complete N × N matrix; it "needs to wait" for it all [00:51:40–00:52:12].
- With online softmax, one row/tile of $Q \cdot K^T/\sqrt{d_k}$ can move directly into the softmax as soon as it is ready; the next row comes and the softmax value is updated again [00:53:04–00:53:47; PDF p6].
- So the softmax always holds a **partial output**: partial output → add more → partial output → … → update done. Then "tile 1 is completed", then tile 2 [00:54:00–00:54:36; PDF p6 "Tile 1 ✓, Tile 2 ✓"].
- This frees HBM first, which is the aim [00:54:36–00:55:20].
- You do not need to keep the scores around once the partial update is done [00:59:50–01:00:29].
- It is used by both FlashAttention and PyTorch SDPA ("Everybody uses, guys") [02:19:30].

5.2 The running state and its terms

In the chunk-by-chunk walkthrough (Topic 6) the tutor named what is tracked for each K chunk [02:32:13–02:37:05; PDF p12 "Flash Attention → Online Softmax Algo"]:

Term (as taught)	Meaning
K-statistics update	The general name for the update that arrives from each K chunk. Chunks $K_1 \ldots K_4$ send updates U_1, U_2, U_3, U_4; the updates are combined (PDF p12: $U_1 + U_2 + U_3 + U_4$).
Running normalization	Keeps track of the **softmax denominator** (the sum of the exponentials, $e^s{}_1 + e^s{}_2 + \ldots$). It keeps getting updated as new chunks come, "close to a moving average but the denominator gets the update" [02:34:00, 02:54:42–02:55:04].
Running output	The partial weighted sum of V that will end up as the final output; "it can be rescaled at any point of time" [02:35:29–02:37:11].

WARNING — Do not apply a plain softmax to each small block

If softmax were applied separately to each 2 × 2 block, each block would be normalised without the context of the other blocks, so the normalisation would be "pretty much incorrect". Normalisation needs knowledge of the other values [02:30:05–02:32:13].

Correction by the tutor later: it is not "softmax at each tile level"; it is an **incremental global softmax update** in which each tile gives a partial update [02:52:36–02:53:18]. (An earlier remark, "first accumulate all values, then apply softmax" [02:35:47], is superseded by this corrected version.)

Q&A

Q: Kumar: online softmax is row by row, but it needs a complete Q·K operation first, right?

A: Yes, you need one complete Q·K combination for the part you update, but only partial softmax values are kept; you never need the whole N × N matrix on top of the memory. [00:58:37–00:59:17]

Q: Murtaza: do we add up the tiles?

A: No. Once the softmax value (partial update) is done, you do not need that tile information and can remove it. Later tiles still need V_1, V_2, V_3... [00:59:50–01:00:37]

Q: Jatin: then how was it correct in FlashAttention (compared with the 2 × 2-block case)?

A: The tutor said the strategy of that softmax is different there; "I'm coming to that part". See the conflict note below. [02:34:25–02:34:42]

Q: Murtaza: do we apply softmax for each tile?

A: No; that intuition is wrong. It computes the equivalent of the global softmax incrementally; it is a partial update, an incremental global softmax update. [02:52:36–02:53:18]

CONFLICT FLAGGED — FA softmax vs MEA softmax

At [02:19:30] and [02:32:22] the tutor says both use the online softmax algorithm and "only the naming convention is different". At [02:34:25] he says "the strategy of that softmax is different" in FlashAttention. At [02:37:11] he calls the MEA description "more sophisticated" while at [02:38:19] the class agrees the two are about 90% similar. Both statements are kept; no resolution was given. The tutor offered to cover the softmax step by step in the next class [02:54:17].

EXTRA CONTEXT (NOT FROM CLASS) — The standard formulation, for beginners

Usual "online softmax" keeps a running maximum m and a running sum l of exponentials. When a new block has a larger maximum, the old partial sums are rescaled by $\exp(m_{old} - m_{new})$ before adding the new block. This is why the result equals the full softmax exactly. The notes above use the tutor's terms; this box adds the usual technical detail.

Material reference: *PDF p6 (online softmax: wait & update; tile 1 ✓ tile 2 ✓), p12 (K-statistics update, running normalization, running output). NB 8: "accumulate softmax in SRAM".*

Additional info from materials (not said in class)

- NB 8 only mentions "accumulate softmax in SRAM" in its diagram; there is no formula in any notebook.

Said in class, not in materials

- Everything in 5.1–5.2: wait-and-update, the three running terms, the warning against per-block softmax, the tile-discarding rule.

6. PyTorch SDPA internals: memory-efficient attention (chunking)

Summary. PyTorch's memory-efficient attention (MEA) is the same idea as FlashAttention with different words: **tiling → chunking**. The tutor worked a full example with shapes, first splitting only the queries, then splitting Q, K and V [02:01:57–02:37:35].

KEY CONCEPT — Tiling vs chunking

FA: break Q, K, V → **tiling**. PyTorch SDPA: break Q, K, V → **chunking** (the "memory-efficient attention chunking" strategy) [00:21:34–00:21:59, 02:03:09–02:03:31]. Because the tutor sees little difference in speed or memory between the two, "the entire discussion … is with respect to your MEA" [02:02:47–02:03:09].

6.1 Step A: break only the queries

8 tokens → 64 scores. Queries are cut into 4 chunks of 2 tokens; keys are kept whole (8 tokens) for now [02:03:40–02:07:59; PDF p7].

Chunk	Query tokens	Keys used	Scores in this chunk
1	1, 2	tokens 1–8	2 × 8 = 16
2	3, 4	tokens 1–8	16
3	5, 6	tokens 1–8	16
4	7, 8	tokens 1–8	16
Total			16 + 16 + 16 + 16 = 64 (after concatenation)

Standard vs chunked attention [02:08:40–02:10:32; PDF p8]:

	Standard	Chunked
Queries processed at a time	8 (the whole sequence)	2
Keys	8	8 (no improvement yet; broken later)
Attention scores held at a time	64 (single block)	16 at a time
Full N × N matrix stored on GPU?	Yes	Avoided
Final output	8 tokens	8 tokens, same shape (no impact on output)

The benefit is memory: "rather than keeping that entire matrix, what about only keeping the output?" The read and write speeds for outputs are much faster than working with the big matrix [02:10:03–02:10:28].

6.2 Step B: shapes (d = 4)

EXAMPLE — Tutor's shape walkthrough, first query chunk [02:11:07–02:18:29; PDF p8–9]

- Q, K, V are each **8 × 4** (8 tokens, dimension 4; so $\sqrt{d_k} = \sqrt{4}$).
- Q is cut into Q_1, Q_2, Q_3, Q_4, each **2 × 4** (tokens 1–2, 3–4, 5–6, 7–8). K and V stay **8 × 4**.
- Attention score $S_1 = Q_1 \cdot K^T = (2 \times 4) \cdot (4 \times 8) =$ **2 × 8** = 16 scores.
- $A_1 = \text{softmax}(S_1 / \sqrt{4}) =$ **2 × 8**.
- $O_1 = A_1 \cdot V = (2 \times 8) \cdot (8 \times 4) =$ **2 × 4**.
- Repeat for $Q_2 \ldots Q_4$: each gives 2 × 4. **$O = \text{Concat}(O_1, O_2, O_3, O_4) = 8 \times 4$**, the same shape as before.

The tutor noted that concatenation of outputs is specific to this chunking description; he said it "doesn't really take place" in the FlashAttention description [02:07:33–02:07:45].

6.3 Step C: break Q, K and V

Now all three are chunked the same way [02:18:46–02:23:19; PDF p10]:

Chunk	Q tokens	K tokens	V tokens
1	1–2	1–2	1–2
2	3–4	3–4	3–4
3	5–6	5–6	5–6
4	7–8	7–8	7–8

- Four query chunks Q_1…Q_4, four key chunks K_1…K_4, four value chunks V_1…V_4, each 2 × 4.
- $S_{11} = Q_1 \cdot K_1^T = (2 \times 4)(4 \times 2) =$ **2 × 2** = 4 scores. Q_1 must also pair with K_2, K_3, K_4, each 2 × 2.
- So Q_1 has 4 blocks × 4 scores = **16 scores**; Q_2, Q_3, Q_4 also 16 each (4 × 4 = 16 combinations in total; the tutor did not write all 16).

How Q_1 becomes O_1 [02:25:33–02:29:40; PDF p11–12]

Query	K chunks	V chunks	Final
Q_1	K_1, K_2, K_3, K_4	V_1, V_2, V_3, V_4	O_1
Q_2	K_1…K_4	same	O_2
Q_3	K_1…K_4	same	O_3
Q_4	K_1…K_4	same	O_4

```
Q1 --K1--> attention score --> V1 \
   --K2--> attention score --> V2  \   combine with online softmax
   --K3--> attention score --> V3  /   updates  --> O1
   --K4--> attention score --> V4 /
Finally: O = Concat(O1, O2, O3, O4)
```

Difference from the FlashAttention description: here each K chunk's score is **immediately paired with the matching V chunk**, and the running softmax state is updated; the tutor said "this is a little bit different from flash attention" in presentation only [02:26:04–02:26:27].

KEY CONCEPT — No loss of information

Mathematically there is no loss of information, because the same Q·K and V products are computed, only in pieces and with a corrected (online) softmax [02:19:16–02:19:25].

The tutor's opinion: the MEA literature is "a bit complex" and slightly more sophisticated than the simple FlashAttention intuition, but there is no major difference [02:36:44–02:37:35].

Q&A

Q: Participant: how do we handle the softmax with chunks?

A: Use the online softmax algorithm; see Topic 5. [02:24:52–02:25:16]

Q: Participant: can we apply softmax to each 2 × 2 block?

A: No; normalisation would be incorrect. See Topic 5 for the corrected statement. [02:30:05–02:32:13]

Material reference: *PDF p6 (SDPA → tiling (chunk) → memory-efficient attention, PyTorch), p7–p12. NB 9 (SDPA does the same math as the manual softmax($QK^T/\sqrt{d}$)V and does not materialise the full matrix).*

Additional info from materials (not said in class)

- NB 9: SDPA "avoids materializing the full attention matrix in memory and avoids Python-level overhead of separate matmul/softmax/matmul calls".
- NB 9 proof of equivalence (max absolute difference between manual and SDPA outputs, causal): shape (1,4,8,16): 2.38e-07; (2,8,32,64): 5.96e-07; (1,2,128,32): 3.58e-07; all within atol = 1e-5.

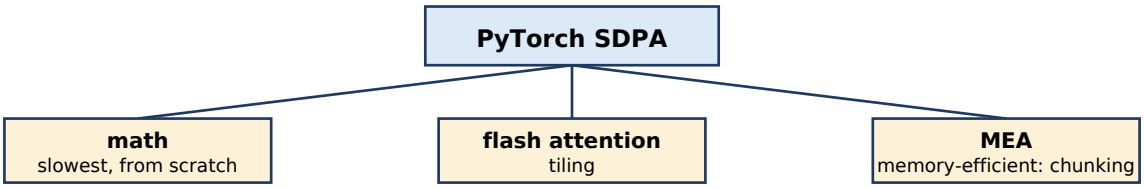
Said in class, not in materials

- The whole 8-token / d = 4 shape walkthrough and the Q/K/V chunk table.

7. The PyTorch SDPA API: backends, FA vs SDPA, causal flag

Summary. `torch.nn.functional.scaled_dot_product_attention` is a single PyTorch call that picks one of three backends depending on your hardware. It does the same math as hand-written attention. FlashAttention is the separate, installable package with the same core idea [02:38:36–02:39:05].

7.1 Three backends



Recreated from PDF p13 ("Pytorch SDPA API": SDPA → math / Flash Attention (Tiling) / MEA (Chunk)).

Backend	What the tutor said	Time
math	The slowest; everything from scratch ("import math and based on that it does the calculation"); nobody uses it. It is the fallback, e.g. on CPU.	[02:39:33–02:39:40, 02:42:44–02:43:29]
flash attention	FlashAttention behind the SDPA call; the tutor identified this backend with the Flash Attention 2 version.	[02:39:40, 01:58:38]
MEA	Memory-efficient attention (chunking); PyTorch's own variant; works only inside PyTorch.	[02:39:47–02:40:32]

- By default PyTorch checks the hardware and picks the best available backend; you can also invoke one manually (e.g. via the SDPA kernel selection) [02:41:17–02:41:46, 01:22:59–01:23:31].
- On a CPU, memory-efficient and flash are "not possible" so it falls back to math [02:43:06–02:43:29].
- For benchmarking, the device should be CUDA; otherwise the math backend is invoked "which we really don't want" [02:46:31–02:46:42].

7.2 FlashAttention vs PyTorch SDPA (MEA)

	FlashAttention (package)	PyTorch SDPA / MEA
Core idea	Tiling + online softmax	Chunking + online softmax
Similarity	Class vote: "close to 90%" the same [02:37:39–02:38:30]	Speed/memory difference about 5–10% in the tutor's tests [02:47:14]
Dependence	Installable separately (pip), works with transformers and other libraries	A dedicated PyTorch API: without PyTorch you cannot use it
Updates	Versions 1, 2, 3, 4; other community implementations exist	Tutor: "in any year till now we have not received any update" [unclear: statement as spoken]
Name use	"Flash attention"	People say "SDPA"; the term MEA is "used very, very less"

TIP — When do you install FlashAttention manually?

If you only use PyTorch, the SDPA backend handles it and you do not need to install anything. If you work with transformers and other libraries outside PyTorch's SDPA, manual installation of FlashAttention matters [01:39:19, 02:42:10–02:42:22]. The tutor added that manual installation is much tougher than using the SDPA flash backend [01:58:34–01:58:47].

7.3 The Hugging Face switch

- When loading a model with transformers (AutoTokenizer / AutoModel), the parameter `attn_implementation` chooses the attention. **"eager" = the standard version**; change it to the flash attention option to use FlashAttention. "This is the only change"; "nobody looks into this parameter, but you should" [01:02:53–01:03:11, 01:38:21–01:39:03].

- The tutor said that in today's world flash attention is "the default … behind the scenes" in many setups [01:41:38–01:41:44; [unclear: context]].

> **EXTRA CONTEXT (NOT FROM CLASS) — Exact string**
>
> In Hugging Face transformers the flash option is passed as `attn_implementation="flash_attention_2"`; the standard one is `"eager"`.

7.4 Causal (decoder) attention

The same function also supports masked/causal attention. One parameter is needed: the tutor said "is causal equal to true" (spoken as "is casual = true"; also called the mask) so it can be used on the decoder side. The rest of the block (concatenating heads, output projection) stays the same [02:43:46–02:44:27, 02:48:30–02:48:54].

```
import torch.nn.functional as F
out = F.scaled_dot_product_attention(q, k, v, is_causal=True)   # from NB 9
```

> **Q&A**
>
> **Q:** Tutor to class: what is the difference between FlashAttention and PyTorch SDPA?
>
> **A:** Class guessed about 90% similar. Answer: SDPA is a dedicated PyTorch API, so it needs PyTorch; FlashAttention can be installed separately. SDPA has three backends and chooses on its own. [02:37:39–02:41:46]

Material reference: *PDF p13; NB 8 mermaid "SDPA call → sdpa_kernel context → flash / efficient / math"; NB 9 recap.*

Additional info from materials (not said in class)

- NB 8 dispatch: CUDA + supported shape → flash; CUDA + unsupported shape → efficient; no GPU support → math fallback. NB 9 additionally lists a **cudnn** backend (math / flash / efficient / cudnn); the tutor named three.
- NB 9: "A migration that changes outputs is a bug"; the notebook's purpose is to prove numerical equivalence, not speed. Every attention variant module in the repo (MHA, MQA, GQA, MLA) already calls SDPA internally.
- NB 9 live-migration check: before/after with is_causal=True gives identical outputs (allclose, atol = 1e-5, result True).

Said in class, not in materials

- FA vs SDPA comparison (~90% similar, PyTorch dependence), the Hugging Face "eager" switch, the hardware-detection fallback, T4 problems.

8. Demos and benchmarks (Colab) and installing FlashAttention

Summary. The tutor ran two Google Colab notebooks (not supplied; numbers below are as read aloud or shown and may be approximate). Lesson: speed and memory gains are small for short sequences and huge for long ones. Installing the FlashAttention package in Colab is slow and fragile.

8.1 FlashAttention demo notebook

- Setup: check PyTorch and CUDA versions (CUDA 13 shown); batch 1, heads, sequence length 2048 (spoken as "208"; later "2048"), head dimension per head [01:20:53–01:21:45].
- Manual function: QK^T → scale → softmax → multiply by V (standard attention) [01:21:51–01:22:06].
- Then the same call through the SDPA **flash backend**; shapes must match "because shape matters" [01:22:37–01:23:48].
- First error: "sdpa_kernel is not defined" (a missed import). Then on **T4** it still fails: unsupported hardware. The tutor switched to **L4**, and it worked [01:24:01–01:25:13].
- Correctness: outputs compared with torch.allclose: approximately equal → True [01:27:07–01:28:12].

Results as read out (L4 GPU)

Test	Manual attention	Flash (SDPA backend)	Note
Single run (seq 2048)	≈ 1.75 ms	≈ 0.14–0.15 ms	Spoken as "PyTorch 0.14 and flash attention 0.15"; which is which is [unclear] [01:31:37]
Seq 512	0.142 ms	0.071 ms	This is ≈ 2×, though the tutor said "close to 10× improvements, always, roughly" [01:32:42]
Seq 2048	1.765 ms	0.149 ms	≈ 11.8× (computed from the two numbers)
Seq 4096	≈ 6 (ms; "increased to 6")	not stated	[01:33:05]
Seq 64,000	OOM (out of memory)	—	Tutor tried 64K after Rushi asked about 1 million [01:33:30–01:34:07]
Memory (peak)	≈ 138 MB	≈ 2 MB	Tutor: "100× I cannot say, but somehow closer" [01:29:41–01:30:19]

KEY CONCEPT — Why the gap grows with sequence length

The tutor: "the entire game is based on sequence length." For small sequences the impact is on the lower end; as you increase the length the main impact appears [01:31:59–01:32:20]. He also said FlashAttention helps in both training and inference, and "always use flash attention" when loading Hugging Face models [01:29:08–01:29:20, 01:31:53].

8.2 SDPA notebook

- Same approach: a manual attention function and a `F.scaled_dot_product_attention` function, random Q, K, V, same shapes; output shapes compared [02:44:45–02:46:23].
- Sequence length 2048 in the first run. The tutor started with T4 here ([02:44:27–02:44:40]); the results below were as read out.
- Speed: manual ≈ 10 ms vs SDPA ≈ 5 ms: "close to 2× speed". Memory: ≈ 520 MB vs ≈ 8 MB [02:46:52–02:47:14].
- FlashAttention vs SDPA: numbers "very, very close", maybe 5–10% difference, depending most on max sequence length [02:47:14–02:47:34].
- At about 8,000 tokens the memory improvement is ≈ 100–200×; speed-up "pretty much the same stuff" [02:47:44–02:48:26].
- Homework hint: try increasing the sequence length, and test causal attention with the is_causal flag [02:46:23–02:46:31, 02:48:30–02:48:54].

8.3 Installing the flash-attn package (what went wrong)

What the tutor tried	Outcome	Time
pip install flash-attn (latest release) on Colab	Took very long; C++ build. Ninja normally makes it 3–5 minutes; Ninja was missing at first.	[01:38:04–01:43:28, 01:52:18]
Install with "no build isolation" (from official docs)	Build took more than 5–6 minutes (tutor expected under 5); then "the build is done". System RAM peaked at ≈ 46–52 GB of 53 GB.	[01:48:54–01:55:35]
Flash Attention 4 on an A100 (CUDA 13, 80 GB)	"No matching distribution found". FA4 is for H100 and B200 only.	[01:56:35–01:59:08]
Student wheel (cp311) from Vignesh	Also an error; unresolved.	[02:00:22–02:01:24]
Planned alternative	Clone the repo and use setup.py manually (not tried); FA3 has a dedicated Hopper folder in the repo.	[01:59:14–01:59:50]

WARNING — Colab installation is fragile

The tutor: setting up FlashAttention in Colab "takes much more time"; the latest release may be slower; check GPU compatibility first. He could not finish the install in class and asked students to try and report back [02:00:03–02:01:52].

Q&A

Q: Rushi: try 1 million tokens.

A: Tutor tried 64,000 on the manual path: it ran out of memory (OOM). 1 million "doesn't run even on H100s"; people use other strategies (e.g. Colibri, [unclear]). [01:33:30–01:34:07, 00:59:17–00:59:50]

Q: Alok / Vignesh: results with other versions or wheels.

A: Alok tested another package (variant 4); Vignesh's wheel failed too. The tutor asked others to test and report. [01:55:38, 02:00:22–02:01:26]

Q: Participant: which runtime did we select?

A: L4 GPU; no need to select it manually in the notebook. [01:27:17–01:27:44]

Material reference: *NB 8 benchmark (different environment, below). The two Colab notebooks and their plots were not supplied.*

Additional info from materials (not said in class)

Backend (NB 8 run on a Mac, MPS device)	Mean latency	Throughput
math	2.059 ms	485.6 calls/s
flash	1.222 ms	818.3 calls/s

- Shape (2, 8, 1024, 64). NB 8 warns that local results are backend/device dependent and should be judged on methodology, not absolute numbers; real CUDA gap widens as sequence length grows.

Said in class, not in materials

- All the Colab results above, T4/L4/A100 compatibility, the install troubles, OOM at 64K.

9. Practical guidance and Q&A (live questions)

Summary. The tutor's advice on how deep to go, how hardware drives the choice of attention implementation, and where newer attention variants live.

9.1 Nilesh: how deep to go on the maths? [02:58:14–03:01:59]

- Nilesh finds the low-level maths hard; he works on inference cost optimisation on AWS Lambda / batch inference at a higher level.
- Tutor: it is a **one-time thing**. Skip if you are not doing engineering work. But "every inference engineer works at the low level".
- If you are on managed cloud services (e.g. AWS serverless training), you can skip much of this; if you build custom/open-source/self-hosted systems, learn it (price differences between managed and self-hosted are large).
- Do it at least once: "everything is based on matrices", not calculus, so not too hard. **Write with pen and paper**; watching the lecture alone is hard to absorb.
- Make your own simple examples (with AI if needed) and keep notes for interviews: "approach is important".
- Nilesh's own takeaway: know the trade-offs (e.g. less memory and faster, but the benefit shows at long sequences).

9.2 sridhar k: production inference and training [03:02:01–03:07:21]

Q&A

Q: How do I decide what to use for inference in production; how do FA vs MEA choices play in?

A: It depends on hardware compatibility first. Example: FlashAttention 4 needs H100 / B100 / B200-class hardware; if you have it, proceed. If the hardware is not there, the choice of algorithm does not matter. Everyone is working with longer sequences, so you should use it where possible. [03:02:40–03:03:16]

Q: So with frontier models we need not worry, only for our own models?

A: Even for training and fine-tuning, specialised kernels are used. Each framework has its own custom, optimised kernels, which work only in that framework. [03:03:28–03:04:04]

Q: How do these factors come in when training a small language model; do we alter parameters at runtime?

A: Will be clearer when we reach that class. Fine-tuning classes will use small models (e.g. Qwen-family 8B or 27B; names garbled, [unclear]) on H100/H200-class hardware, not L4, because kernels need compatible hardware. In Axolotl, the kernel choice is a YAML-file entry. This comes in Module 4 (parallel to fine-tuning for the first project). [03:04:07–03:05:35]

Q: Is this the latest and greatest, or is there still efficiency work on attention?

A: After MLA there are more: YaRN (as heard), KDA (Kimi Delta Attention) and gated delta attention; "the stack keeps on increasing". Gated delta is asked about in interviews. See Sebastian Raschka's write-up on attention variants (name garbled). [03:06:20–03:07:20; also 00:10:47–00:11:01]

9.3 Kernels and the wider ecosystem (tangents)

- Before kernel engineering, learn the GPU basics (the tutor, to Sanjit) [01:42:18–01:42:24].
- Most frameworks have built their own kernels: Liger, CUDA and Triton kernels were named [00:56:28–00:56:44, 01:42:24–01:42:33].
- Triton is good for TensorRT deployments (tested on a Jetson-type "Xavier"); works well on NVIDIA-compatible hardware; Intel/AMD are trickier [01:43:33–01:44:14].
- FlashAttention is the starting point for kernel work; more specialised kernels do much more of the break-down [01:30:19–01:30:38, 02:49:37–02:49:46].
- Newer attention methods after RoPE: ALiBi, YaRN; the tutor's current favourite is KDA (Kimi Delta Attention) [00:10:47–00:11:01].

Material reference: *Transcript only. Nothing in this section comes from the supplied materials, and it is all said in class.*

10. Key takeaways

1. **Not about accuracy.** FlashAttention and SDPA compute the same attention; they only reduce GPU memory traffic (training and inference).
2. **The problem is N².** Attention scores form an N × N table per head per batch item; doubling the sequence length makes memory 4×; 32K and longer lead to OOM.
3. **HBM is the bottleneck.** Standard attention writes and reads the big matrix to/from HBM twice (after QK^T and after softmax). Peak bandwidth and available HBM are the numbers to check.
4. **Tiling = break Q, K, V into blocks.** An 8 × 8 matrix (64 scores) becomes four 4 × 4 tiles (16 scores each); each query block must still meet every key block.
5. **Online softmax makes tiling work.** "Wait and update": keep a running denominator and a running output; do not softmax each block separately; the result equals a global softmax computed incrementally.
6. **Tiles are not kept.** Once a tile's partial update is done, its scores are discarded; only the running state and later V blocks are needed.
7. **SDPA = chunking (MEA).** The same idea named differently: 4 query chunks of 2 × 4 give $O_1 \ldots O_4$ of 2 × 4 each, concatenated to 8 × 4; the output shape never changes.
8. **SDPA has three backends** (math, flash, memory-efficient) and picks one by hardware; math is the slow fallback. SDPA needs PyTorch; FlashAttention can be installed separately.
9. **Hardware decides.** FA2 is the widely used version; FA3 is for H100 (Hopper); FA4 for H100/B200; T4 is unsupported. If the hardware is missing, the algorithm choice does not matter.
10. **Gains grow with sequence length.** In the tutor's runs: ≈ 10× speed and ≈ 138 MB → 2 MB memory on L4; ≈ 2× speed and 520 MB → 8 MB in the SDPA notebook; 64K tokens OOM on the manual path.

11. Glossary

Term	Meaning
Attention score	One number for a (query, key) pair: how much a token attends to another.
SDPA (formula)	Scaled dot-product attention: softmax($QK^T/\sqrt{d_k}$)·V.
SDPA (PyTorch API)	F.scaled_dot_product_attention, a single fused call that picks a backend.
Q, K, V	Query, key, value matrices (rows = tokens, columns = dimension).
d_k	Dimension of keys; scores are divided by $\sqrt{d_k}$ (the tutor used 4 in the shape example).
Softmax	Turns scores into weights that sum to 1: each e^s divided by the sum of all e^s in the row.
Online softmax	Softmax computed incrementally with a running denominator and output ("wait and update").
Running normalization	The running softmax denominator kept while blocks arrive.
Running output	The partial weighted-V result updated per block.
Tile / tiling	A small block of the score matrix and the process of splitting work into such blocks (FlashAttention term).
Chunk / chunking	The same idea in PyTorch's memory-efficient attention.
FA / FlashAttention	Tiling-based attention by Tri Dao et al.; versions 1–4.
MEA	Memory-efficient attention: PyTorch's chunking-based SDPA backend.
HBM	High bandwidth memory: the GPU's main memory (VRAM) on enterprise GPUs; holds weights and gradients.
GDDR	Graphics memory used on consumer/lower-end GPUs (L4 has GDDR6).
SRAM / shared memory	Fast on-chip static RAM (about 200 KB per SM); where tiles are processed.
Registers	Per-thread storage on a GPU.
L1 / L2 cache	Fast small caches between SMs and memory.
SM	Streaming multiprocessor: main processing unit of an NVIDIA GPU (H100: about 132).
CUDA cores	Units that do the arithmetic; SIMD = single instruction, multiple data.
Memory bandwidth	How fast data can move to/from memory (H100 ≈ 3.35 TB/s, L4 ≈ 300 GB/s).
OOM	Out of memory error.
FP16 / FP32	16-bit / 32-bit floating point: 2 / 4 bytes per number.
Eager	Hugging Face attn_implementation value meaning standard attention.
Kernel	A specialised GPU routine (CUDA, Triton, Liger) that implements an operation efficiently.
Causal attention	Decoder-style attention with a mask; enabled by the is_causal flag.
Backend (SDPA)	One of math, flash, memory-efficient that SDPA dispatches to.
Ninja	Build tool needed to compile flash-attn quickly.
GPU families	Ampere, Hopper, Ada, Blackwell (Rubin upcoming); older T4 (2018) is unsupported for FlashAttention.

12. Action items, logistics, resources

12.1 Homework and next steps

- **Try to install FlashAttention** (FA2 minimum) in your GPU environment and report to the tutor "by tomorrow"; spend 1–2 hours or use an agent in a GPU environment; the tutor will also try [02:00:03–02:01:52, 01:42:49].
- Increase the sequence length in both notebooks (e.g. 4096, 8000) and observe time and memory; try causal attention with is_causal=True [02:46:23, 02:48:30].
- Add the two notebook links to Notion (the tutor adds them too) and keep Notion bookmarked [02:49:46–02:50:40, 00:12:36].
- Run git pull for the newly pushed MLA and other notebooks (MLA push was missing earlier) [00:09:07–00:10:30].
- Write the maths by hand and keep a personal note for interviews [03:00:25–03:01:04].
- Optional reading: the FlashAttention papers (Tri Dao), Sebastian Raschka's attention-variants write-up (gated delta, KDA) [01:37:03, 03:06:59–03:07:17].

12.2 Schedule and logistics (as stated)

- **Tomorrow:** last theory class of Module 3: scaling laws (Kaplan, Chinchilla). The tutor will also revisit the softmax step in detail if needed, and Divesh joins as he will take LangChain [02:50:10–02:50:18, 02:51:26, 02:54:17, 03:05:39–03:06:17].
- **Next week:** LangChain starts (Divesh; about four easy-notebook classes; RAG is covered step by step through it). Based on last week's poll; people from the RAG batch can take one month off; Advanced-batch people who were not in the RAG batch attend [00:06:31–00:07:24, 00:11:58–00:12:17, 01:44:14–01:44:41, 02:51:26–02:51:57].
- Order afterwards: finish RAG module, then the fine-tuning module (Module 4, tied to the first project), later agents; vLLM will be covered with deployment [00:07:00–00:07:13, 00:13:03–00:13:19, 03:05:22–03:05:33].
- Tangents: PyTorch conference on 20–21 Oct; an "agent arena" hackathon link to be shared; several team members travelling that week [02:55:15–02:57:45].

12.3 Resources, links and tools mentioned

Resource	Notes
Course GitHub repo and Notion page	Same link as before; notebooks and links are on Notion [00:11:11, 00:12:19]
Google Colab notebooks (FlashAttention demo and SDPA demo)	Shared in chat as viewer links; not supplied here. Runtimes tried: T4 (fails), L4, A100, H100 [01:19:21–01:19:38, 02:49:03–02:49:24]
PyPI `flash-attn` and Dao AI Lab GitHub	Official package; Ninja build; "no build isolation" option [01:36:12–01:37:35, 01:43:17]
Hugging Face transformers	attn_implementation parameter (eager vs flash attention) [01:38:21]
Excalidraw	Used for the tutor's flow diagrams (not supplied) [01:02:25]
Diagram tools	Diagram Designs, Design Creator, and one called "Archifi" [unclear: spelling] [00:07:33–00:07:52, 02:51:03]
Sebastian Raschka's attention variants	Name heard as "Sebastian Rashfas Blow" [unclear]; gated delta, KDA [03:06:59]
Colibri (GitHub)	Mentioned for running bigger models on smaller GPUs [unclear: exact name] [00:59:24–00:59:50]
Kernels / frameworks	Triton, Liger, CUDA, TensorRT, Axolotl (YAML kernel selection) [00:56:28, 01:43:33, 03:05:02]

13. Unresolved or unclear points

- **1 GB vs 64 MB:** the tutor's spoken "close to 1 GB" for batch 2, 8 heads, sequence 1024 [01:15:45] does not match the PDF (≈ 64 MB for FP32); see Topic 2.
- **FA vs MEA softmax:** statements that they are the same (naming only) vs "the strategy is different" [02:19:30, 02:34:25]; the tutor promised a softmax step-by-step tomorrow.
- **"Unified memory" remark** [01:09:50]: the tutor agreed that SDPA/this is "unified memory" in response to a chat comment; the materials do not support or explain it.
- **MEA updates:** "in any year till now we have not received any update" [02:40:56–02:41:17]; unclear which project this refers to.
- **Benchmark details:** 4096-length flash timing not stated; single-run labels (0.14 vs 0.15) ambiguous; "close to 10×" at seq 512 vs measured ≈ 2×; memory ratio said "close to 100×" but numbers give ≈ 69× (138/2, computed).
- **CUDA compute-capability statement** for FlashAttention [00:57:22–00:57:37] ("more than 30 … less than 13") garbled; actual threshold not stated clearly.
- **SM counts for L4** and exact H100 table values were given from memory/AI table; not verified.
- **"Default is flash attention behind the scenes"** [01:41:44]: unclear which library or setting.
- **Whether the FlashAttention-install problem** was the latest release, memory, or Ninja: unresolved; students to test.
- **Not supplied:** both Colab notebooks, the Excalidraw diagrams, the AI-generated H100/L4 table, and chat messages.
- **PagedAttention** was not taught; only NB 10 contains material (Appendix A).

Appendix A. PagedAttention / vLLM (from NB 10 only; not taught in class)

Everything in this appendix comes from the supplied notebook. The tutor only said vLLM and the practical part will be covered during deployment [00:07:00–00:07:13].

KEY CONCEPT — Problem and fix

Problem: if output length is unknown, a naive server preallocates a buffer for max_len per request. Real lengths vary, so most of it goes unused (internal fragmentation), limiting concurrent requests.

Fix (PagedAttention, Kwon et al. 2023, vLLM): split the cache into fixed-size blocks (like OS pages); allocate blocks on demand; a per-sequence **block table** maps logical positions to physical blocks. Waste per request is at most block_size − 1 tokens instead of max_len − actual_length. It changes the allocation strategy, not the attention math.

NB 10 simulation (CPU, not the real vLLM kernel): 32 requests, max_len = 2048, block_size = 16, 24 layers, 8 KV heads, head_dim 128:

Allocation strategy	Total allocated	Wasted vs actually used
naive (preallocate max_len per request)	6.00 GB	2.54 GB
paged (block_size-rounded)	3.49 GB	24.38 MB
actually used (lower bound)	3.46 GB	—

Result: ≈ 41.9% less allocated than naive. Sweeping block size: 8 → 3.47 GB (waste 10.88 MB); 16 → 3.49 GB (24.38 MB); 32 → 3.51 GB (46.88 MB); 64 → 3.56 GB (100.88 MB); 128 → 3.63 GB (172.88 MB). Trade-off: smaller blocks waste less but need more block-table bookkeeping.

```
Logical view (per sequence)         Physical KV blocks (shared pool)
tokens 0-15  ----block table---->   block 7
tokens 16-31 ----block table---->   block 2
tokens 32-47 ----block table---->   block 9
```

The real vLLM install and throughput benchmark vs Hugging Face generate() is in a separate GPU Colab notebook (per NB 10).

Quick revision cheat sheet

Formula	SDPA = softmax($QK^T/\sqrt{d_k}$)·V. S = QK^T; P = softmax($S/\sqrt{d_k}$); O = PV.
Why a problem	N tokens → N × N scores ($O(N^2)$). Batch 2, 8 heads, seq 1024 ≈ 16 M scores ≈ 32 MB (FP16) / 64 MB (FP32). 2048 ≈ 256 MB, 4096 ≈ 1 GB. **Double the length = 4× memory.** 32K → OOM.
Goal of FA/SDPA	Same output, less memory and GPU traffic. NOT an accuracy gain. Helps training and inference.
Previous topics	KV cache: reuse K,V. MQA/GQA: fewer KV heads. MLA: compact cache. FA: execute attention with less memory.
GPU memory	HBM = big main memory (weights, gradients, slow). SRAM/shared memory (~200 KB per SM) = tiny, fast. SM = streaming multiprocessor (H100 ≈ 132). H100: 80 GB, ≈ 3.35 TB/s; L4: 24 GB, ≈ 300 GB/s. Check peak bandwidth + HBM size.
Standard attention	QK^T (N × N) → write HBM → read → scale + softmax → write HBM → read → ×V. Two big HBM trips.
Tiling (FA)	Break Q, K, V into blocks. 8 × 8 = 64 scores → 4 tiles of 4 × 4 = 16. $S_{11}=Q_1K_1^T/\sqrt{d_k}$, S_{12}, S_{21}, S_{22}. Each query block meets every key block. Load tile into SRAM → partial scores → online softmax → accumulate V → loop; write only output.
Online softmax	"Wait and update": running denominator (running normalization) + running output; K-statistics updates $U_1 \ldots U_4$. **Never softmax each block alone**; incremental global softmax. Discard tiles after update.
Chunking (MEA)	8 tokens, d = 4: Q,K,V = 8 × 4. Q_i: 2 × 4. $S_1 = Q_1K^T$ = (2×4)(4×8) = 2 × 8 (16 scores); A_1 = 2 × 8; $O_1 = A_1V$ = 2 × 4. O = Concat($O_1 \ldots O_4$) = 8 × 4. With K,V chunked: S_{11} = 2 × 2; Q_1 has 16 scores.
SDPA backends	math (slowest, CPU fallback) / flash (tiling) / MEA (chunking). PyTorch auto-picks by hardware; needs CUDA for the fast ones. is_causal=True for decoders. FA ≈ 90% same as SDPA; FA is a separate pip package.
FA versions	FA2 = standard; FA3 = H100 (Hopper); FA4 = H100/B200; T4 unsupported. Hugging Face: attn_implementation "eager" → flash.
Benchmarks	L4: 1.765 → 0.149 ms (seq 2048); memory 138 → 2 MB; 64K manual = OOM. SDPA notebook: ≈ 10 → 5 ms; 520 → 8 MB. Gap grows with sequence length.
Next	Test flash-attn install; scaling laws tomorrow; LangChain next week.